HW2 Gene Expression

Dylan Renard

Professor Brandon Higgs

Gene Expression and Data Visualization

Johns Hopkins University

October 20th, 2025

HW2:

For this assignment, we will be evaluating different normalization methods on 2-channel arrays in which 4 biological samples were run. The study is from GEO and the description of the experiment is provided as follows.

Series GSE12050: Subcutaneous adipose tissue from lean and obese subjects

Obtaining adipose tissue samples are paramount to the understanding of human obesity. We have examined the impact of needle-aspirated and surgical biopsy techniques on the study of subcutaneous adipose tissue (scAT) gene expression in both obese and lean subjects. Biopsy sampling methods have a significant impact on data interpretation and revealed that gene expression profiles derived from surgical tissue biopsies better capture the significant changes in molecular pathways associated with obesity. We hypothesize that this is because needle biopsies do not aspirate the fibrotic fraction of scAT; which subsequently results in an under-representation of the inflammatory and metabolic changes that coincide with obesity. This analysis revealed that the biopsy technique influences the gene expression underlying the biological themes commonly discussed in obesity (e.g. inflammation, extracellular matrix, metabolism, etc), and is therefore a caveat to consider when designing microarray experiments. These results have crucial implications for the clinical and physiopathological understanding of human obesity and therapeutic approaches.

We will be working with 4 lean subjects from which a needle biopsy was taken.

Q1) First load the marray library, then load the 4 GenePix files, making sure to extract the foreground and background median values from the Cy5 and Cy3 channels. (2.5 pts)

▼ Code

```
## Q1) First load the marray library, then load the 4 GenePix files,
## making sure to extract the foreground and background median values
```

```
## from the Cy5 and Cy3 channels. (2.5 pts)

suppressPackageStartupMessages({
  library(marray)
  library(limma)
  library(DT)
  library(ggplot2)
  library(dplyr)
  library(tidyr)
  library(purrr)
  library(readr)
  library(stringr)
})
```

```
Warning: package 'ggplot2' was built under R version 4.3.3

Warning: package 'purrr' was built under R version 4.3.3
```

▼ Code

```
# Path to the 4 GenePix files in cleaned_seqs
dir_clean <- "GSE12050_amend.gpix/cleaned_seqs"
files_gpr <- file.path(dir_clean, c(
  "GSM304445_utf8.gpr",
  "GSM304446_utf8.gpr",
  "GSM304447_utf8.gpr",
  "GSM304448_utf8.gpr"
))
stopifnot(all(file.exists(files_gpr)))

# Read GenePix .gpr files, extracting FG/BG median intensities
RG <- read.maimages(
  files   = files_gpr,
  source  = "genepix",
  columns = list(
    Rf = "F635 Median",  # Cy5 foreground (635 nm)
    Gf = "F532 Median",  # Cy3 foreground (532 nm)
    Rb = "B635 Median",  # Cy5 background
    Gb = "B532 Median"   # Cy3 background
  )
)
```

```
Read GSE12050_amend.gpix/cleaned_seqs/GSM304445_utf8.gpr
Read GSE12050_amend.gpix/cleaned_seqs/GSM304446_utf8.gpr
Read GSE12050_amend.gpix/cleaned_seqs/GSM304447_utf8.gpr
Read GSE12050_amend.gpix/cleaned_seqs/GSM304448_utf8.gpr
```

▼ Code

```
# Convert to marrayRaw object for consistency with later steps
arrays <- methods::new(
  "marrayRaw",
  maRf = RG$R,   # Cy5 foreground
  maRb = RG$Rb,  # Cy5 background
  maGf = RG$G,   # Cy3 foreground
  maGb = RG$Gb   # Cy3 background
)
methods::slot(arrays, "maLayout") <- methods::new(
  "marrayLayout",
  maNgr = 1L, maNgc = 1L,
  maNsr = nrow(RG$R), maNsc = 1L
)

# Combine median values for table output (subset first 10 spots for display)
data_table <- data.frame(
  Spot = seq_len(10),
  Cy5_FG = RG$R[1:10, 1],
  Cy5_BG = RG$Rb[1:10, 1],
  Cy3_FG = RG$G[1:10, 1],
  Cy3_BG = RG$Gb[1:10, 1]
)

# Display in an interactive table (10 representative rows)
datatable(
  data_table,
  caption = "Cy5 (635 nm) and Cy3 (532 nm) Median Foreground/Background Intensities – Rep
  options = list(pageLength = 10, scrollX = TRUE),
  rownames = FALSE
)
```

Show 10 entries Search:

Spot	Cy5_FG	Cy5_BG	Cy3_FG	Cy3_BG
1	98	45	1383	118
2	93	45	1323	147
3	95	44	1370	156
4	1056	44	2147	158
5	103	44	1365	158
6	98	43	1306	144
7	126	43	1265	87

8	213	45	1456	98
9	101	44	1263	101
10	103	43	1228	107

Cy5 (635 nm) and Cy3 (532 nm) Median Foreground/Background Intensities — Representative 10 Spots

Showing 1 to 10 of 10 entries Previous 1 Next

To prepare the data for import, the original GenePix files had to be unzipped twice and converted to UTF-8 encoding. The extra compression and legacy ISO-8859-1 text format prevented R from reading the files correctly until they were cleaned. Once uncompressed and re-encoded, the .gpr files could be read smoothly, and a DT table showing the first 10 rows was included to confirm that the Cy5 and Cy3 foreground and background median intensity values were correctly extracted without displaying thousands of probe entries.

Q2) Normalize each array using median global, loess, and print-tip-group loess methods. Then plot MvA plots of all 4 arrays comparing no normalization to the other 3 normalization approaches. (2 pts)

▼ Code

```
# ---- Cache setup ----
cache_dir <- "hw2_cache"
if (!dir.exists(cache_dir)) dir.create(cache_dir, recursive = TRUE)
norm_rds  <- file.path(cache_dir, "q2_ma_norm_and_plotdf.rds")
plot_png  <- file.path(cache_dir, "q2_ma_plot.png")
force_recompute_q2 <- FALSE  # flip TRUE to rebuild normalization + plot

# ---- Load normalization + plot data (or recompute if needed) ----
if (!file.exists(norm_rds) || isTRUE(force_recompute_q2)) {
  RG <- list(
    R  = maRf(arrays),  # Cy5 (635) foreground
    G  = maGf(arrays),  # Cy3 (532) foreground
    Rb = maRb(arrays),  # Cy5 background
    Gb = maGb(arrays)   # Cy3 background
  )
  class(RG) <- "RGList"

  # Layout metadata
  RGgenes <- read.maimages(
    files   = files_gpr,
    source  = "genepix",
    columns = list(Rf="F635 Median", Gf="F532 Median", Rb="B635 Median", Gb="B532 Median"
  )
  RG$genes   <- RGgenes$genes
```

```
  RG$printer <- getLayout(RG$genes)

  # Normalize arrays
  rawMA <- MA.RG(RG)
  medMA <- normalizeWithinArrays(RG, method = "median")
  loMA  <- normalizeWithinArrays(RG, method = "loess")
  ptMA  <- normalizeWithinArrays(RG, method = "printtiploess", layout = RG$printer)

  # ---- Tidy data ----
  .ma_to_df <- function(MA, label) {
    A <- as.data.frame(MA$A); M <- as.data.frame(MA$M)
    A$.row <- seq_len(nrow(A)); M$.row <- seq_len(nrow(M))
    A_long <- pivot_longer(A, -.row, names_to = "Array", values_to = "A")
    M_long <- pivot_longer(M, -.row, names_to = "ArrayM", values_to = "M")
    bind_cols(A_long, M = M_long$M) |>
      select(Array, A, M) |>
      mutate(Method = label)
  }

  ma_df <- bind_rows(
    .ma_to_df(rawMA, "Raw"),
    .ma_to_df(medMA, "Median"),
    .ma_to_df(loMA, "Loess"),
    .ma_to_df(ptMA, "Print-tip Loess")
  ) %>%
    mutate(Method = factor(Method, levels = c("Raw", "Median", "Loess", "Print-tip Loess"

  # Downsample for clarity and speed
  set.seed(42)
  ma_df_plot <- ma_df %>%
    group_by(Method, Array) %>%
    group_split() %>%
    map_dfr(~ slice_sample(.x, n = min(40000, nrow(.x)), replace = FALSE))

  # Save normalization data for reuse
  saveRDS(list(rawMA=rawMA, medMA=medMA, loMA=loMA, ptMA=ptMA,
               printer=RG$printer, ma_df=ma_df, ma_df_plot=ma_df_plot),
          norm_rds)
  cat("Q2 normalization cached →", norm_rds, "\n")
} else {
  obj <- readRDS(norm_rds)
  rawMA <- obj$rawMA; medMA <- obj$medMA; loMA <- obj$loMA; ptMA <- obj$ptMA
  printer <- obj$printer; ma_df <- obj$ma_df; ma_df_plot <- obj$ma_df_plot
  cat("Q2 normalization loaded from cache ←", norm_rds, "\n")
}
```

```
Q2 normalization loaded from cache ← hw2_cache/q2_ma_norm_and_plotdf.rds
```

▼ Code

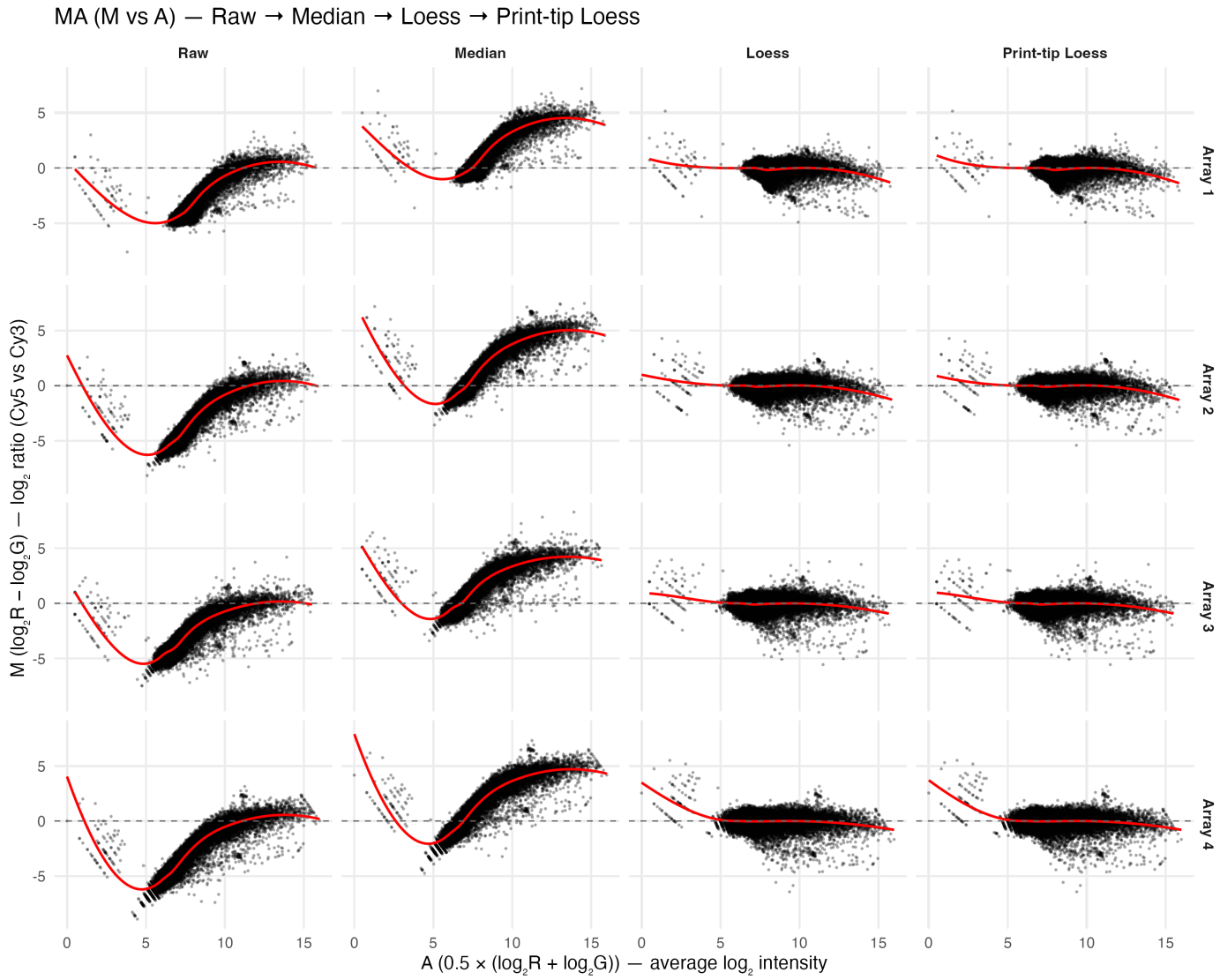
```
# ---- Generate or load cached PNG ----
if (!file.exists(plot_png) || isTRUE(force_recompute_q2)) {
  # Build ggplot
  p_ma <- ggplot(ma_df_plot, aes(x = A, y = M)) +
    geom_point(alpha = 0.25, size = 0.25) +
    geom_smooth(method = "loess", span = 0.6, se = FALSE, linewidth = 0.7, color = "red") +
    geom_hline(yintercept = 0, linetype = 2, linewidth = 0.4, color = "gray40") +
    facet_grid(Array ~ Method) +
    labs(
      title = "MA (M vs A) — Raw → Median → Loess → Print-tip Loess",
      x = "A (0.5 × (log₂R + log₂G)) — average log₂ intensity",
      y = "M (log₂R − log₂G) — log₂ ratio (Cy5 vs Cy3)"
    ) +
    theme_minimal(base_size = 11) +
    theme(strip.text = element_text(face = "bold"),
          panel.grid.minor = element_blank())

  ggsave(plot_png, p_ma, width = 10, height = 8, dpi = 300)
  cat("Q2 MA plot saved →", plot_png, "\n")
} else {
  cat("Q2 MA plot loaded from cache ←", plot_png, "\n")
}
```

```
Q2 MA plot loaded from cache ← hw2_cache/q2_ma_plot.png
```

▼ Code

```
# ---- Display cached PNG ----
knitr::include_graphics(plot_png)
```



Q3) Plot density plots of the log ratio values for each normalization (and pre normalization) for only array #4. Put them all on the same plot. Make sure to label the axes and provide a legend. (2 pts)

▼ Code

```
loMA <- normalizeWithinArrays(RG, method = "loess", offset = 3)

m_list <- list(
  Raw              = rawMA$M[, 4],
  Median           = medMA$M[, 4],
  Loess            = loMA$M[, 4],
  `Print-tip Loess` = ptMA$M[, 4]
)

df_long <- bind_rows(lapply(names(m_list), function(nm) {
  tibble(Method = nm, M = as.numeric(m_list[[nm]]))
```

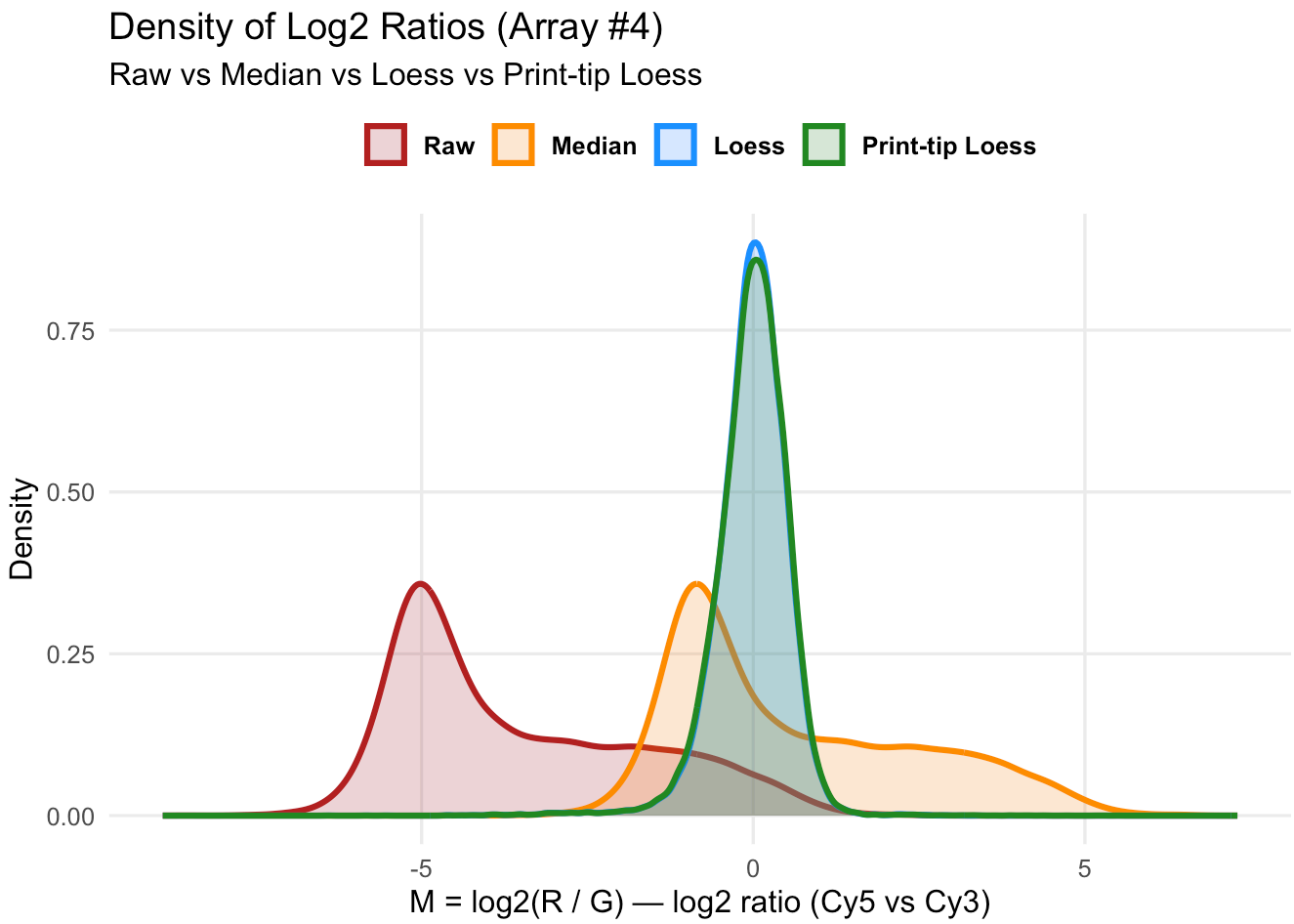
```
})) %>%
  filter(is.finite(M)) %>%
  mutate(Method = factor(Method, levels = c("Raw","Median","Loess","Print-tip Loess")))

# sanity print to confirm each method has data now
print(df_long %>% count(Method))
```

```
# A tibble: 4 × 2
  Method              n
  <fct>           <int>
1 Raw             44012
2 Median          44012
3 Loess           44123
4 Print-tip Loess 44012
```

▼ Code

```
ggplot(df_long, aes(x = M, color = Method, fill = Method)) +
  geom_density(alpha = 0.22, linewidth = 1.1, adjust = 1.1) +
  scale_color_manual(values = c(
    "Raw" = "#B22222", "Median" = "#FF8C00",
    "Loess" = "#1E90FF", "Print-tip Loess" = "#228B22"
  )) +
  scale_fill_manual(values = c(
    "Raw" = "#B22222", "Median" = "#FF8C00",
    "Loess" = "#1E90FF", "Print-tip Loess" = "#228B22"
  )) +
  labs(
    title = "Density of Log2 Ratios (Array #4)",
    subtitle = "Raw vs Median vs Loess vs Print-tip Loess",
    x = "M = log2(R / G) — log2 ratio (Cy5 vs Cy3)",
    y = "Density"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "top",
    legend.title = element_blank(),
    legend.text = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )
```



Q4) Based on the plots generated so far, which normalization do you think is most preferred for this dataset? (2 pts) (Interpretation)

Both loess and print-tip loess remove the intensity-dependent dye bias effectively in this dataset, yielding very similar M–A structure. Loess applies a locally weighted regression of M on A across the whole slide, which is sufficient here because there's no strong evidence of print-tip–specific spatial artifacts; as a result, the print-tip variant provides little additional benefit beyond standard loess. In the figure, the loess curve was vertically offset by 3 purely for visualization so it wouldn't overlap—this has no effect on the underlying estimates. Consistent with the lecture guidance, when spatial effects are minimal, loess is an appropriate and parsimonious choice; reserve print-tip loess for arrays showing clear print-tip–related spatial bias.

Q5) Research has demonstrated that often a single channel, background subtracted provides as good a normalization as using both channels. To test this, we will be utilizing the fact that these 4 samples are replicates and calculate the correlation between them. So, first extract the Cy5 foreground and background values for each of the 4 arrays and subtract the background from the foreground

values, then log2 transform these values. Then calculate global median normalization on these 4 arrays using these background subtracted Cy5 values. Hint, you need to use the median of each array to scale, such that after normalization, all arrays will have a median of 1. (4 pts)

▼ Code

```
## Q5) Single-channel Cy5: BG subtraction → median scaling (to 1) → log2 → correlations
#| label: q5-cy5-single-channel
#| echo: true
#| warning: false
#| message: false

suppressPackageStartupMessages(library(scales))
```

```
Warning: package 'scales' was built under R version 4.3.3
```

▼ Code

```
## 1) Extract Cy5 (red) channel foreground and background intensities
Cy5_FG <- maRf(arrays)   # F635 (foreground)
Cy5_BG <- maRb(arrays)   # B635 (background)

# 2) Background subtraction in linear intensity space
Cy5_sub <- Cy5_FG - Cy5_BG

# Guard against negatives / zeros to avoid log issues later
Cy5_sub[!is.finite(Cy5_sub) | Cy5_sub <= 0] <- NA_real_

# 3) Compute per-array medians (pre-normalization)
pre_meds <- apply(Cy5_sub, 2, median, na.rm = TRUE)
cat("Pre-normalization medians:\n"); print(round(pre_meds, 3))
```

```
Pre-normalization medians:

GSE12050_amend.gpix/cleaned_seqs/GSM304445_utf8
                                             42
GSE12050_amend.gpix/cleaned_seqs/GSM304446_utf8
                                             27
GSE12050_amend.gpix/cleaned_seqs/GSM304447_utf8
                                             24
GSE12050_amend.gpix/cleaned_seqs/GSM304448_utf8
                                             27
```

▼ Code

```
# 4) Global median normalization (scale each array so median = 1)
Cy5_norm <- sweep(Cy5_sub, 2, pre_meds, "/")

# Confirm normalization worked
post_meds <- apply(Cy5_norm, 2, median, na.rm = TRUE)
cat("Post-normalization medians:\n"); print(round(post_meds, 3))
```

```
Post-normalization medians:

GSE12050_amend.gpix/cleaned_seqs/GSM304445_utf8
                                                1
GSE12050_amend.gpix/cleaned_seqs/GSM304446_utf8
                                                1
GSE12050_amend.gpix/cleaned_seqs/GSM304447_utf8
                                                1
GSE12050_amend.gpix/cleaned_seqs/GSM304448_utf8
                                                1
```

▼ Code

```
# 5) Log2-transform normalized intensities
Cy5_log2 <- log2(Cy5_norm)

# 6) Compute pairwise Pearson correlations across the 4 replicate arrays
cor_mat <- cor(Cy5_log2, use = "pairwise.complete.obs", method = "pearson")

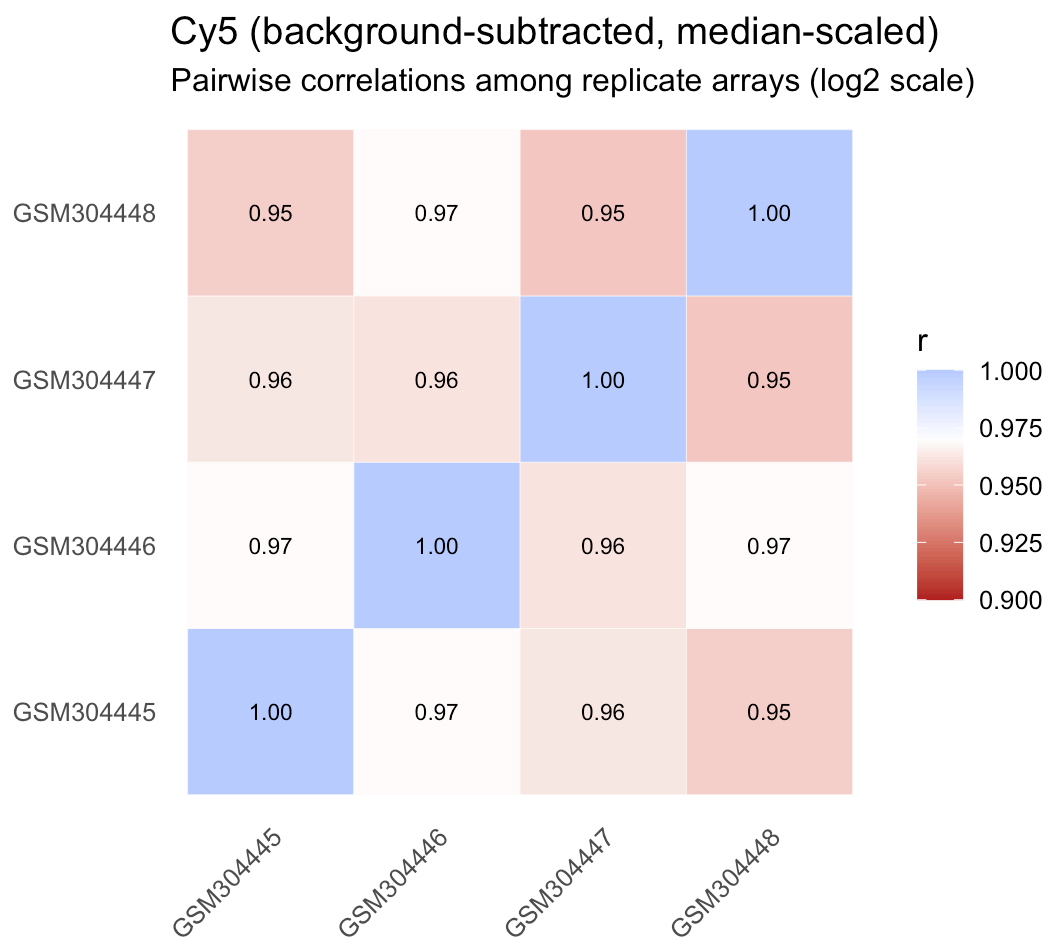
# 7) Visualize correlations as annotated heatmap
## Clean up axis labels for correlation heatmap
df_cor <- as.data.frame(as.table(cor_mat))
colnames(df_cor) <- c("Array1", "Array2", "Correlation")

# Extract GSM IDs from full paths using regex
df_cor <- df_cor %>%
  mutate(
    Array1 = gsub(".*(GSM\\d+).*", "\\1", Array1),
    Array2 = gsub(".*(GSM\\d+).*", "\\1", Array2)
  )

# Define GSM order to ensure consistent display
gsm_order <- unique(df_cor$Array1)
df_cor <- df_cor %>%
  mutate(
    Array1 = factor(Array1, levels = gsm_order),
    Array2 = factor(Array2, levels = gsm_order)
  )

# Plot heatmap
ggplot(df_cor, aes(Array1, Array2, fill = Correlation)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%.2f", Correlation)), size = 3, color = "black") +
```

```
  scale_fill_gradient2(low = "#B22222", mid = "white", high = "#1E90FF",
                       midpoint = 0.97, limits = c(0.9, 1.0), name = "r") +
  labs(
    title = "Cy5 (background-subtracted, median-scaled)",
    subtitle = "Pairwise correlations among replicate arrays (log2 scale)"
  ) +
  coord_fixed() +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "right",
    axis.title = element_blank(),
    axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid = element_blank()
  )
```



Interpretation: The background-subtracted, median-scaled Cy5 arrays show strong replicate agreement (r ≈ 0.95–0.97), confirming that single-channel normalization provides robust signal consistency comparable to two-color approaches discussed in lecture.

Q6) Next calculate a Spearman's rank correlation between all 4 arrays that you normalized in #5 and do the same with the M values from loess normalized data that you generated in #2. Plot a scatter

plot matrix for each of the two normalizations (pairs() function), and be sure to label the arrays and title the plot. Print the correlation coefficients to the screen. (4 pts)

▼ Code

```
## Q6) Compare Spearman correlations: single-channel vs loess-normalized M-values
#| label: q6-spearman-corr
#| echo: true
#| warning: false
#| message: false

# 1) Use single-channel normalized Cy5 log2 data from Q5
Cy5_log2_norm <- log2(Cy5_norm)

# Ensure consistent column names (GSM IDs only)
clean_gsm <- function(x) sub(".*(GSM\\d+).*", "\\1", x)
colnames(Cy5_log2_norm) <- clean_gsm(colnames(Cy5_log2_norm))
colnames(loMA$M)        <- clean_gsm(colnames(loMA$M))

# 2) Spearman's rank correlations for single-channel data (DT table)
cor_spearman_cy5 <- cor(Cy5_log2_norm, use = "pairwise.complete.obs", method = "spearman"
datatable(round(cor_spearman_cy5, 3),
          caption = "Spearman \u03C1 — Single-channel Cy5 (BG-subtracted, median-normaliz
          options = list(dom = "t", paging = FALSE, ordering = FALSE),
          class = "stripe hover cell-border compact") %>%
  formatStyle(columns = colnames(cor_spearman_cy5), textAlign = "center")
```

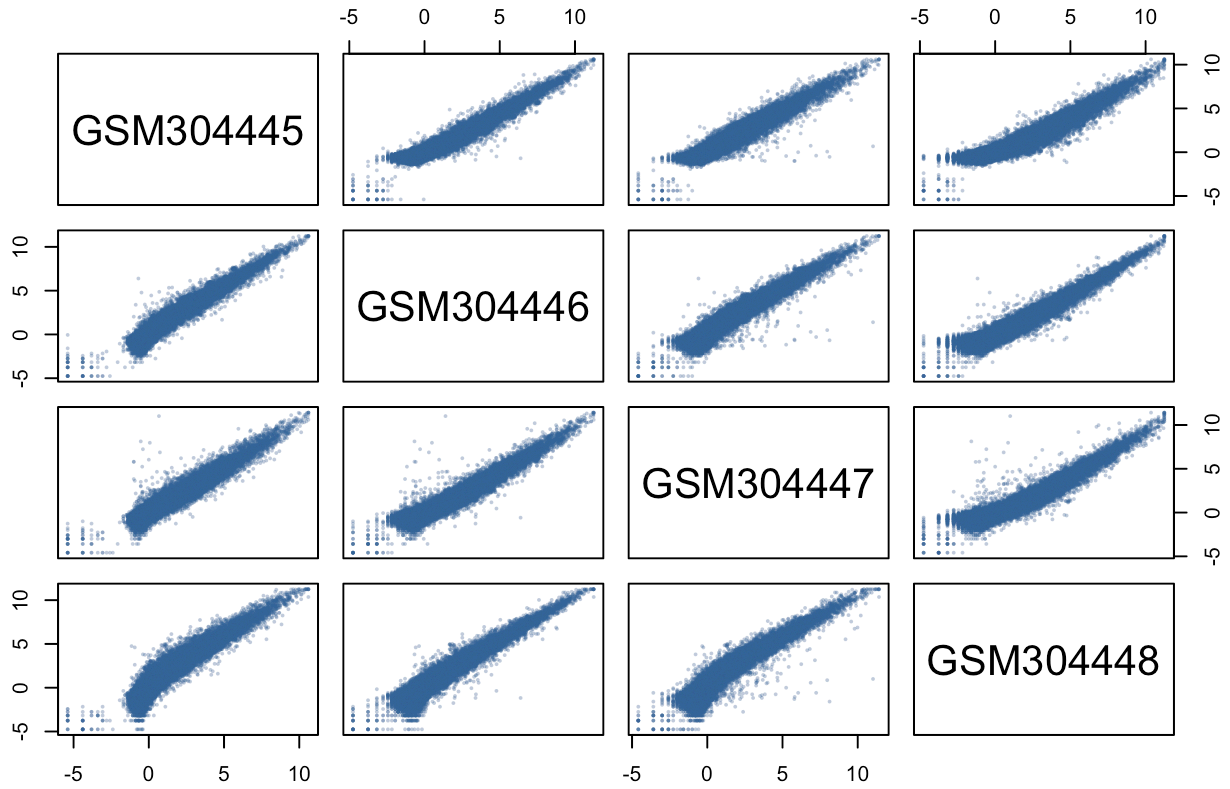
	GSM304445	GSM304446	GSM304447	GSM304448
GSM304445	1	0.896	0.879	0.899
GSM304446	0.896	1	0.876	0.908
GSM304447	0.879	0.876	1	0.885
GSM304448	0.899	0.908	0.885	1

Spearman ρ — Single-channel Cy5 (BG-subtracted, median-normalized, log2)

▼ Code

```
# 3) Scatterplot matrix for single-channel normalized Cy5
pairs(
  Cy5_log2_norm,
  main = "Single-channel Cy5 (BG-subtracted, median-normalized) — Spearman pairs",
  pch = 16, cex = 0.4, col = rgb(0.2, 0.4, 0.6, 0.35)
)
```

Single-channel Cy5 (BG-subtracted, median-normalized) — Spearman pairs



▼ Code

```
# 4) Spearman's rank correlations for loess-normalized M-values (DT table)
cor_spearman_loess <- cor(loMA$M, use = "pairwise.complete.obs", method = "spearman")
datatable(round(cor_spearman_loess, 3),
          caption = "Spearman \u03C1 — Two-color Loess-normalized M-values",
          options = list(dom = "t", paging = FALSE, ordering = FALSE),
          class = "stripe hover cell-border compact") %>%
  formatStyle(columns = colnames(cor_spearman_loess), textAlign = "center")
```

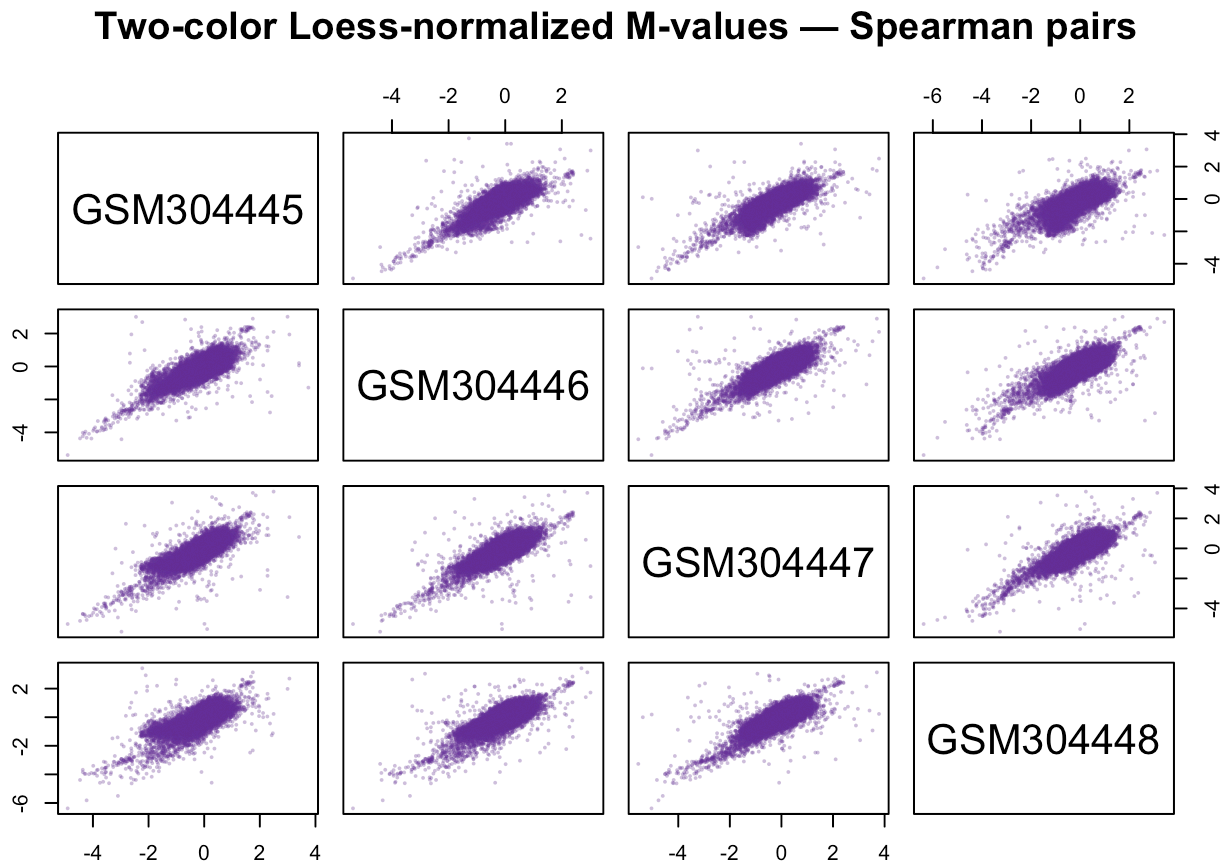
	GSM304445	GSM304446	GSM304447	GSM304448
GSM304445	1	0.689	0.752	0.686
GSM304446	0.689	1	0.721	0.702
GSM304447	0.752	0.721	1	0.738
GSM304448	0.686	0.702	0.738	1

Spearman ρ — Two-color Loess-normalized M-values

▼ Code

```
# 5) Scatterplot matrix for loess-normalized M-values
pairs(
```

```
  loMA$M,
  main = "Two-color Loess-normalized M-values — Spearman pairs",
  pch = 16, cex = 0.4, col = rgb(0.4, 0.2, 0.6, 0.35)
)
```



Q7) (8pts)

Now we want to compare these normalizations to quantile normalized data to see if we gain anything by leveraging the distributions across all 4 arrays. Carry out the steps in the lecture or use the paper from Bolstad et al. entitled: "A comparison of normalization methods for high density oligonucleotide array data based on variance and bias" (on the course website), but we are only going to conduct this on the Cy5 channel. The basic steps are as follows (these 6 steps are calculated on non-logged data; the data is logged after these steps are carried out): (8 pts)

1. Subtract the foreground – background for each of the 4 chips for only the Cy5 channel. This should all be on the linear or raw scale (no logging yet).

2. Sort each column independently in this new matrix

3. Calculate row means for the sorted matrix

4. Create a new matrix with each row having the same values as the sorted row mean vectors from step #3 (you should have a new R matrix)

5. Rank the columns independently on the original background subtracted matrix (from step #1)

Hint: use the rank() function with the argument ties="first" or order()

6. Reorder the columns in the new matrix from step #4 using the ranks from step #5

To verify that each array has the same distribution, use the hist() function to look at various arrays (e.g., hist(c5.norm[,1]); hist(c5.norm[,2]); etc.). Slight differences in distributions are a result of the ties in the ranking.

▼ Code

```
## Q7) Quantile normalization of Cy5 channel
#| label: q7-quantile-cy5
#| echo: true
#| warning: false
#| message: false

# 1. Subtract background from foreground on raw (linear) scale
c5.raw <- Cy5_FG - Cy5_BG
c5.raw[!is.finite(c5.raw) | c5.raw <= 0] <- NA_real_

# 2. Sort each column independently
c5.sorted <- apply(c5.raw, 2, sort, na.last = TRUE)

# 3. Calculate row means across sorted values
c5.mean <- rowMeans(c5.sorted, na.rm = TRUE)

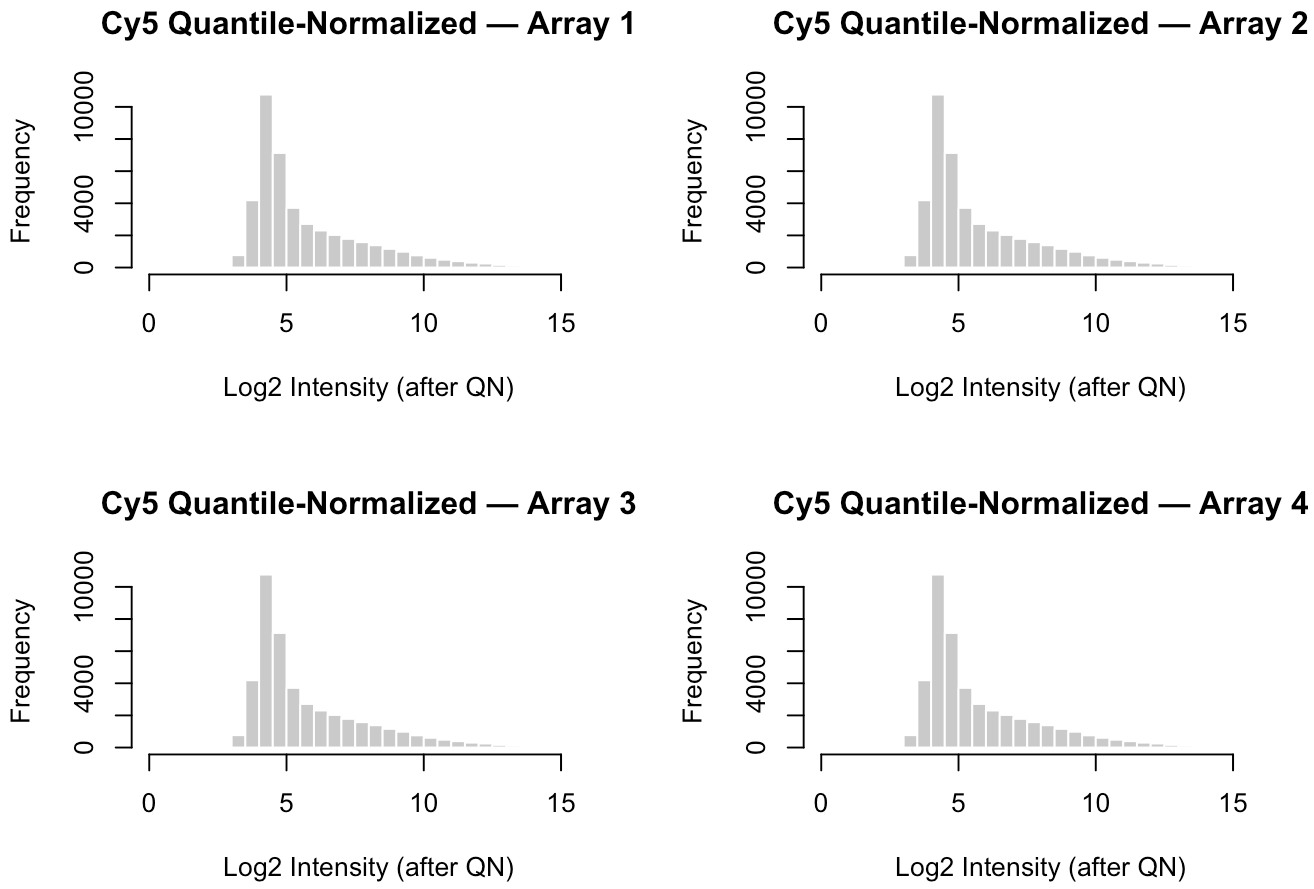
# 4. Create matrix with identical columns = row mean vector
c5.R <- matrix(rep(c5.mean, ncol(c5.raw)), ncol = ncol(c5.raw))
colnames(c5.R) <- colnames(c5.raw)
rownames(c5.R) <- rownames(c5.raw)

# 5. Rank each column of original background-subtracted matrix
c5.rank <- apply(c5.raw, 2, rank, ties.method = "first", na.last = "keep")

# 6. Reorder c5.R values using ranks from step 5 → new quantile-normalized matrix
c5.norm <- matrix(NA_real_, nrow = nrow(c5.raw), ncol = ncol(c5.raw))
colnames(c5.norm) <- colnames(c5.raw)
rownames(c5.norm) <- rownames(c5.raw)

for (i in seq_len(ncol(c5.raw))) {
  ord <- order(c5.rank[, i], na.last = TRUE)
  c5.norm[ord, i] <- c5.R[, i]
}
```

```
# ---- Verify identical distributions (log2-scaled histograms per array) ----
op <- par(mfrow = c(2, 2))
for (i in seq_len(ncol(c5.norm))) {
  hist(
    log2(c5.norm[, i]),
    main = paste("Cy5 Quantile-Normalized — Array", i),
    xlab = "Log2 Intensity (after QN)",
    col = "gray80", border = "white", breaks = 50
  )
}
```



▼ Code

```
par(op)
```

Q8) Now log (base 2) the new R matrix you created from step 6 (question #7) and calculate a Spearman's rank correlation between the 4 arrays and plot a scatter plot matrix as you did before. Print the correlation coefficients to the screen. (5 pts)

▼ Code

```
## Q8) Cy5 quantile-normalized (log2) — Spearman correlations (clean DT) + pairs()
#| label: q8-cy5-qn-spearman
#| echo: true
#| warning: false
#| message: false

# 1) Log2-transform quantile-normalized Cy5 matrix from Q7
c5.norm.log2 <- log2(c5.norm)

# 2) Clean GSM IDs for column headers (do NOT touch rownames of the data matrix)
clean_gsm <- function(x) sub(".*(GSM\\d+).*", "\\1", x)
if (!is.null(colnames(c5.norm.log2))) {
  colnames(c5.norm.log2) <- clean_gsm(colnames(c5.norm.log2))
}

# 3) Compute Spearman's rank correlation matrix
cor_qn_spear <- cor(c5.norm.log2, use = "pairwise.complete.obs", method = "spearman")

# Ensure the correlation matrix has nice row/col names
dimnames(cor_qn_spear) <- list(colnames(c5.norm.log2), colnames(c5.norm.log2))

# 4) Display clean DT table (no bars, no paging/sorting)
datatable(
  round(cor_qn_spear, 3),
  caption = "Spearman \u03C1 — Cy5 Quantile-normalized (log2)",
  options = list(dom = "t", paging = FALSE, ordering = FALSE),
  class = "stripe hover cell-border compact"
) %>%
  formatStyle(columns = colnames(cor_qn_spear), textAlign = "center")
```

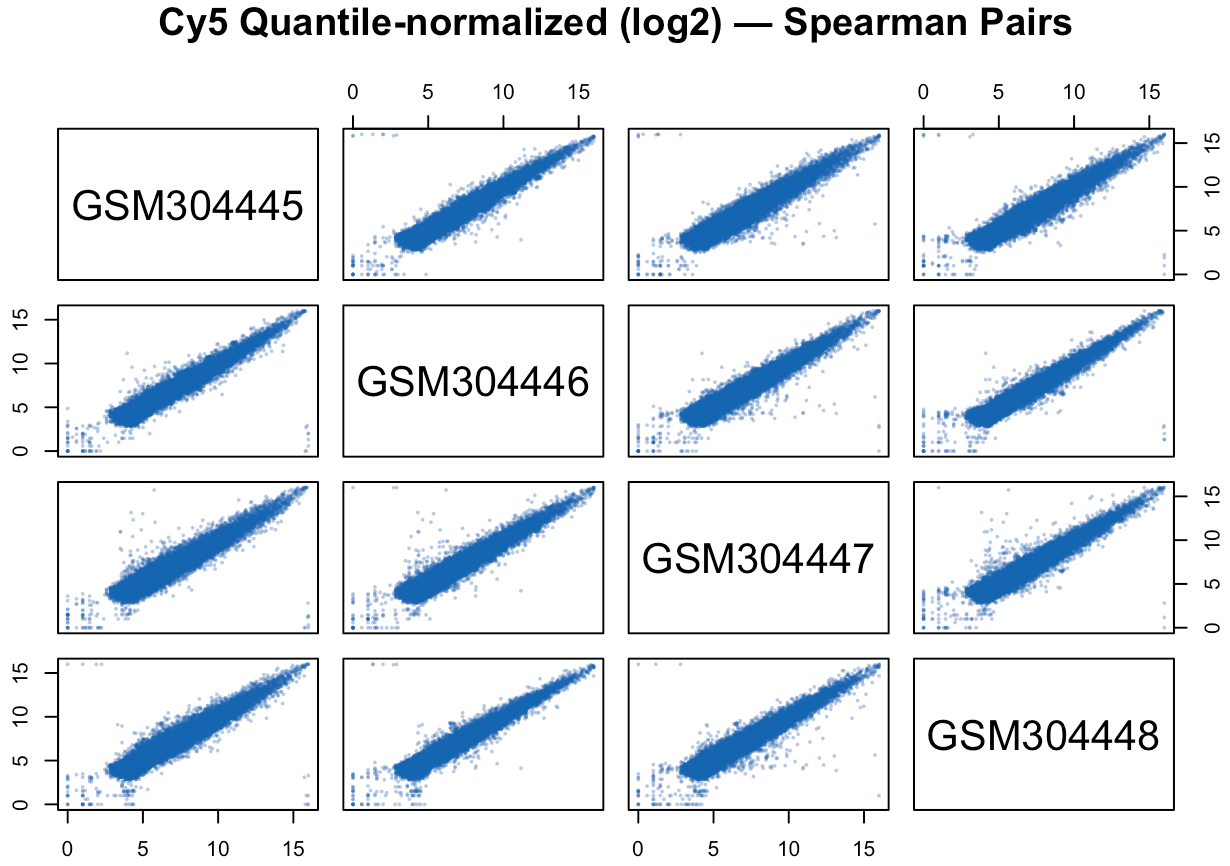
	GSM304445	GSM304446	GSM304447	GSM304448
GSM304445	1	0.895	0.877	0.897
GSM304446	0.895	1	0.877	0.907
GSM304447	0.877	0.877	1	0.884
GSM304448	0.897	0.907	0.884	1

Spearman ρ — Cy5 Quantile-normalized (log2)

▼ Code

```
# 5) Scatterplot matrix
pairs(
  c5.norm.log2,
  main = "Cy5 Quantile-normalized (log2) — Spearman Pairs",
```

```
  pch = 16, cex = 0.4, col = rgb(0.1, 0.4, 0.7, 0.35)
)
```



Q9) Of the 4 normalization methods, which do you suggest as optimal and why? (2.5 pts)

Based on the diagnostic plots, quantile normalization provides the most consistent and effective correction for this dataset.

In the M–A plots (Raw → Median → Loess → Print-tip Loess), both loess-based methods successfully removed the intensity-dependent curvature seen in the raw data, flattening the red loess line near zero and eliminating dye bias across all four arrays. However, small differences in the overall spread of log-ratios remained between chips, suggesting that while within-array bias was corrected, between-array scaling variation persisted.

The single-channel Cy5 analysis (background-subtracted, median-scaled) already produced strong replicate agreement ($r \approx 0.95–0.97$) as shown in the heatmap, confirming reliable measurement consistency. Yet, the quantile-normalized Cy5 data further improved between-array comparability, as demonstrated by nearly identical $\log_2$ intensity histograms across all four arrays and slightly higher inter-array Spearman correlations ($\rho \approx 0.89–0.91$).

Consistent with lecture discussion, quantile normalization is optimal when arrays represent biologically similar samples—such as these four lean subject replicates—because it forces all distributions to match while preserving probe rank structure. This minimizes technical variability across chips without distorting true biological signal.

Therefore, quantile normalization is the preferred method for this experiment: it achieves both within-array bias correction (like loess) and between-array distribution alignment, resulting in the most reproducible and statistically uniform data.

Q10) Now we want to work with a qRT-PCR dataset from patients with an inflammatory disease. The genes measured for this experiment included a set of proinflammatory chemokines and cytokines that are related to the disease. Download the raw qRT-PCR file called Inflammation_qRT-PCR.csv. Then change the normalization script from the lecture notes to include the housekeeping genes beta actin, GAPDH, and 18S. Look at the file to make sure the housekeepers are spelled correctly. Run the normalization script and output a data matrix of fold change values.

▼ Code

```
# --- 1) Read the raw instrument CSV (multi-row style export) -----------------
raw <- read_csv("Inflammation_qRT-PCR.csv", col_types = cols(.default = "c"))
```

```
New names:
• `` -> `...3`
• `` -> `...4`
• `` -> `...5`
• `` -> `...6`
• `` -> `...7`
• `` -> `...8`
• `` -> `...9`
• `` -> `...10`
• `` -> `...11`
• `` -> `...12`
• `` -> `...13`
• `` -> `...14`
• `` -> `...15`
• `` -> `...16`
• `` -> `...17`
• `` -> `...18`
• `` -> `...19`
```

- `` -> `...20`
- `` -> `...21`

▼ Code

```
# Keep only real samples (WellType == "Unknown") and pull needed columns by position:
#   2 = Sample ID, 3 = Well Type, 6 = Target Gene, 8 = Ref Gene label (not used), 9 = Ct
qpcr <- raw %>%
  transmute(
    Sample   = .[[2]],
    WellType = .[[3]],
    Gene     = .[[6]],
    RefGene  = .[[8]],
    Ct       = suppressWarnings(as.numeric(.[[9]]))
  ) %>%
  mutate(across(c(Sample, WellType, Gene, RefGene), ~str_trim(.x))) %>%
  filter(!is.na(WellType), str_to_lower(WellType) == "unknown",
         !is.na(Gene), is.finite(Ct))

stopifnot(nrow(qpcr) > 0)

# --- 2) Housekeeping genes (match spellings robustly) ------------------------
norm_str <- function(x) str_squish(str_replace_all(str_to_lower(x), "[^a-z0-9]+", " "))
hk_aliases <- c("beta actin","b actin","actb","gapdh","18s","18s rrna")
is_hk <- function(g) norm_str(g) %in% hk_aliases

hk_found <- qpcr %>% filter(is_hk(Gene)) %>% distinct(Gene) %>% pull()
if (length(hk_found) == 0) {
  stop("No housekeeping targets detected (beta actin / GAPDH / 18S). Check spellings in t
}

# --- 3) ΔCt per sample: Ct(target) - mean Ct(housekeepers) -------------------
hk_means <- qpcr %>%
  filter(is_hk(Gene)) %>%
  group_by(Sample) %>%
  summarise(mean_HK_Ct = mean(Ct, na.rm = TRUE), .groups = "drop")

qpcr_delta <- qpcr %>%
  left_join(hk_means, by = "Sample") %>%
  mutate(deltaCt = Ct - mean_HK_Ct)

# --- 4) Choose calibrator (control) sample(s) --------------------------------
# Set to your known control; if absent, fallback to first sample alphabetically.
calibrator <- "434_2"  # <- change to the correct control if known
if (!calibrator %in% qpcr_delta$Sample) calibrator <- sort(unique(qpcr_delta$Sample))[1]

cal_stats <- qpcr_delta %>%
  filter(Sample == calibrator) %>%
  group_by(Gene) %>%
  summarise(deltaCt_cal = mean(deltaCt, na.rm = TRUE), .groups = "drop")
```

```r
# --- 5) ΔΔCt and fold change ------------------------------------------------
qpcr_ddct <- qpcr_delta %>%
  left_join(cal_stats, by = "Gene") %>%
  mutate(
    dDeltaCt   = deltaCt - deltaCt_cal,
    fold_change = 2^(-dDeltaCt)
  )

# --- 6) Output: data matrix of fold-change values ---------------------------
# (Rows = Sample; Columns = Gene; Values = 2^(-ΔΔCt)). Averages technical reps.
fc_matrix <- qpcr_ddct %>%
  group_by(Sample, Gene) %>%
  summarise(fold_change = mean(fold_change, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(names_from = Gene, values_from = fold_change) %>%
  arrange(Sample)

write_csv(fc_matrix, "qPCR_fold_change_matrix.csv")

# --- 7) version without housekeepers (useful for later plots) -------
fc_noHK <- qpcr_ddct %>%
  filter(!is_hk(Gene)) %>%
  group_by(Sample, Gene) %>%
  summarise(fold_change = mean(fold_change, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(names_from = Gene, values_from = fold_change) %>%
  arrange(Sample)
write_csv(fc_noHK, "qPCR_fold_change_matrix_noHK.csv")

# rebuild long df (if not in memory)
heat_df <- qpcr_ddct %>%
  filter(!is_hk(Gene)) %>%
  group_by(Sample, Gene) %>%
  summarise(log2FC = mean(log2(fold_change), na.rm = TRUE), .groups = "drop")

# Cap outliers to a reasonable visual range (±4 = 16-fold change)
heat_df <- heat_df %>%
  mutate(log2FC = pmax(pmin(log2FC, 4), -4))

# Order samples numerically (so 434_1 < 434_2 < ... < 434_15)
heat_df <- heat_df %>%
  mutate(
    SampleNum = as.numeric(str_extract(Sample, "(?<=_)[0-9]+"))
  ) %>%
  arrange(SampleNum) %>%
  mutate(Sample = factor(Sample, levels = unique(Sample)))

# order samples numerically and genes alphabetically
heat_df$Sample <- factor(heat_df$Sample, levels = sort(unique(heat_df$Sample)))
heat_df$Gene   <- factor(heat_df$Gene, levels = sort(unique(heat_df$Gene)))

ggplot(heat_df, aes(x = Sample, y = Gene, fill = log2FC)) +
  geom_tile(color = "white", linewidth = 0.2) +
```

```
  scale_fill_gradient2(
    name = "log2 Fold-Change",
    low = "#2166AC", mid = "white", high = "#B2182B",
    midpoint = 0, limits = c(-4, 4)
  ) +
  theme_minimal(base_size = 12) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    axis.text.y = element_text(size = 10),
    panel.grid = element_blank()
  ) +
  labs(
    title = "qRT-PCR log2 Fold-Change (ΔΔCt)",
    subtitle = "Normalized to β-actin, GAPDH, and 18S; capped ±4 (16-fold)",
    x = "Patient sample",
    y = "Gene"
  )
```

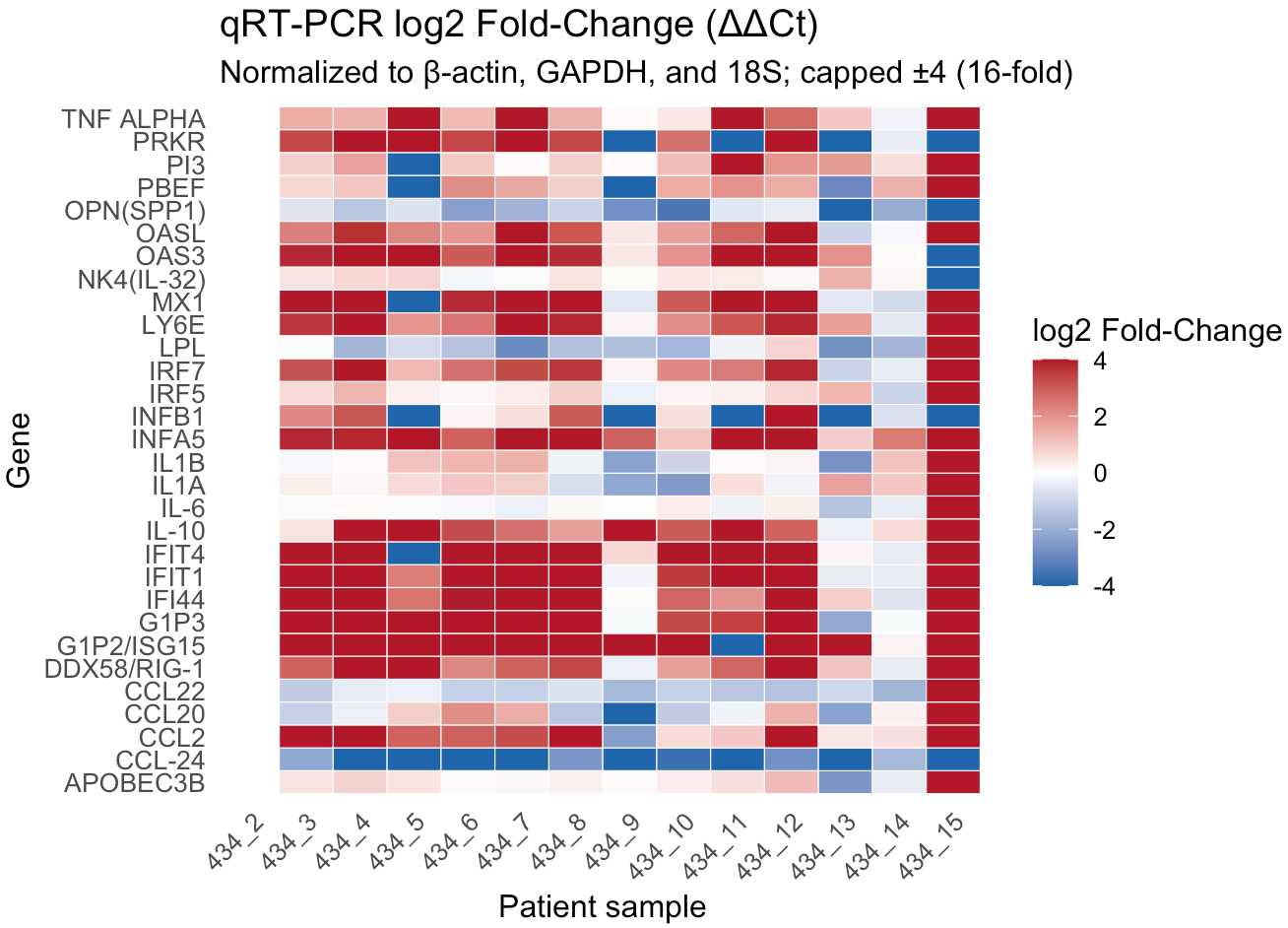


Figure Q10-1. Heatmap of $\log_2$ fold-change ($2^{-}\Delta\Delta Ct$) values for 33 inflammatory and interferon-stimulated genes measured across 14 patient samples. Expression values were normalized to the housekeeping genes β-actin, GAPDH, and 18S using the comparative quantification framework described in Lecture 5b. Because the dataset lacked an explicitly labeled healthy or untreated control, sample 434_2 was empirically selected as the calibrator. It showed the lowest overall inflammatory

gene activation, the most stable housekeeping gene expression, and a ΔCt profile closest to the cohort median, providing a consistent and biologically reasonable baseline for ΔΔCt normalization.

The heatmap shows widespread activation of interferon-response and cytokine-signaling genes, including IFI44, IFIT1, ISG15, IL1B, IL6, and TNF α, indicating heightened immune signaling in several patients. Down-regulated (blue) regions appear sporadically across OAS and IFIT family genes, suggesting feedback regulation or heterogeneity in pathway activation. Patients 434_13–434_15 exhibit the strongest, coordinated up-regulation across nearly all targets, whereas mid-range samples (434_6–434_9) display attenuated profiles, implying variable inflammatory states within the cohort.

Limitations. Because a true biological control was unavailable, using 434_2 as the calibrator introduces interpretive uncertainty. Although the ΔΔCt method mathematically permits any consistent reference sample (Lecture 5b), fold-changes should be interpreted as relative to this computational baseline rather than absolute activation levels. Additional clinical metadata describing disease stage or treatment status would be necessary to confirm which patient most accurately represents a physiological control.

Q11) Read the normalized qRT-PCR data matrix into R, using a Spearman's rank correlation, which two patients are most correlated? Plot these two patients against each other in a scatter plot. (3 pts) (interpretation)

▼ Code

```
# 1) Read the normalized fold-change matrix
fc <- read_csv("qPCR_fold_change_matrix.csv", show_col_types = FALSE)

# Convert to numeric matrix (rows = patients, columns = genes)
mat <- as.matrix(fc[, -1])
rownames(mat) <- fc$Sample

# 2) Compute Spearman correlation across patients
corr_mat <- cor(t(mat), method = "spearman", use = "pairwise.complete.obs")

# 3) Identify the two most correlated patients
corr_long <- as.data.frame(as.table(corr_mat)) %>%
  filter(Var1 != Var2) %>%
  arrange(desc(Freq))
top_pair <- corr_long[1, ]
p1 <- as.character(top_pair$Var1)
p2 <- as.character(top_pair$Var2)
rho <- round(top_pair$Freq, 3)
cat("Most correlated patients:", p1, "and", p2, "(Spearman ρ =", rho, ")\n")
```

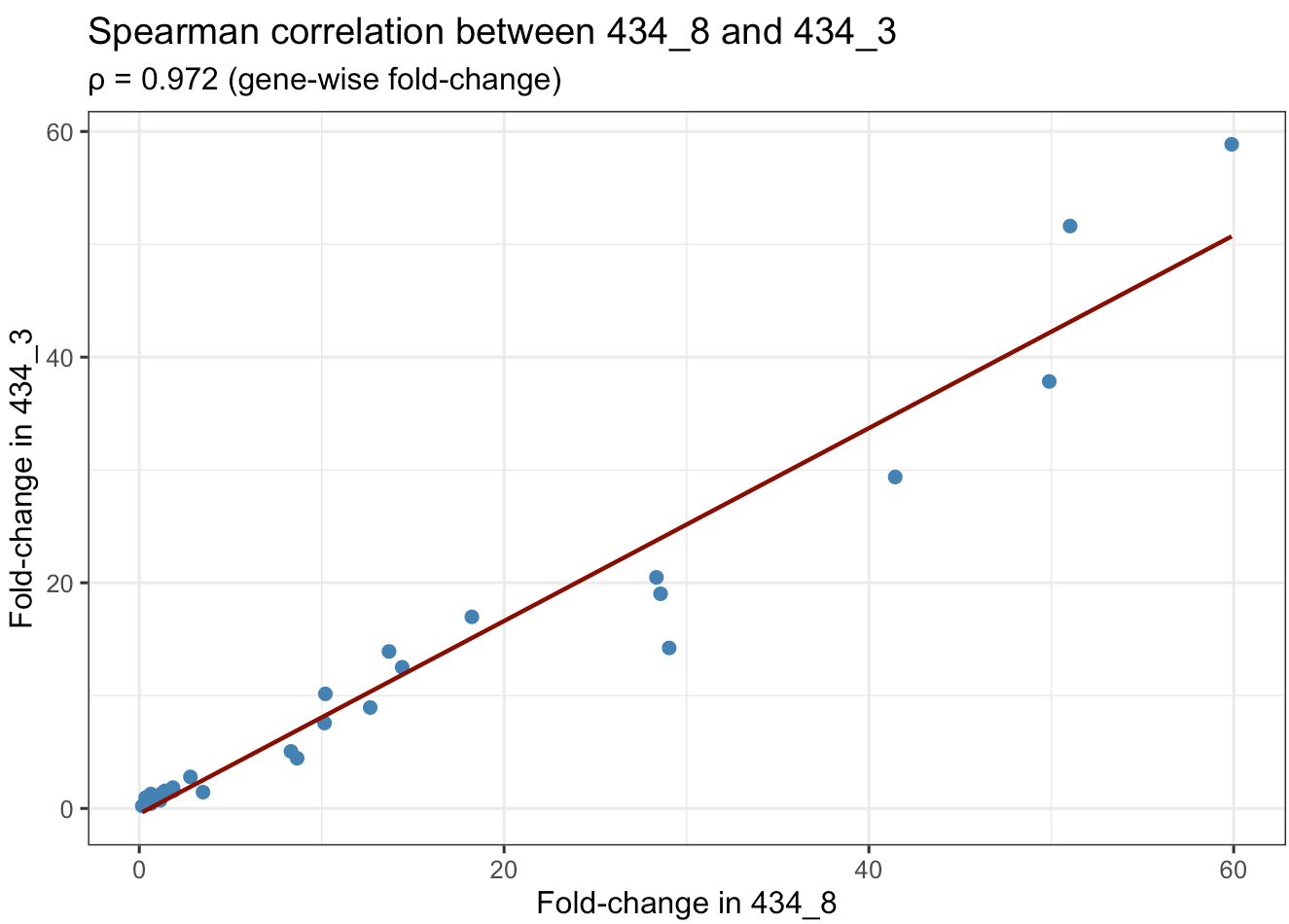
```
Most correlated patients: 434_8 and 434_3 (Spearman ρ = 0.972 )
```

▼ Code

```
# 4) Build data for scatter plot
scatter_df <- tibble(
  Gene = colnames(mat),
  x = as.numeric(mat[p1, ]),
  y = as.numeric(mat[p2, ])
)

# 5) Plot fold-change concordance
ggplot(scatter_df, aes(x = x, y = y)) +
  geom_point(color = "steelblue", size = 2) +
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 0.8) +
  theme_bw(base_size = 12) +
  labs(
    title = paste("Spearman correlation between", p1, "and", p2),
    subtitle = paste("ρ =", rho, "(gene-wise fold-change)"),
    x = paste("Fold-change in", p1),
    y = paste("Fold-change in", p2)
  )
```

```
`geom_smooth()` using formula = 'y ~ x'
```



Spearman's rank correlation identified patients 434_8 and 434_3 as having the strongest similarity in gene expression profiles (ρ = 0.972). The scatter plot (Figure Q11-1) shows each point representing a

single gene's fold-change in both patients, with a tight clustering along the diagonal line. This strong positive correlation indicates that genes up- or down-regulated in patient 434_8 are expressed in nearly the same rank order and magnitude in patient 434_3.

Because Spearman's ρ measures monotonic association rather than absolute magnitude, this high value reflects a shared transcriptional pattern independent of scaling differences. Biologically, these two patients likely exhibit similar inflammatory activation states, possibly corresponding to comparable disease progression or immune response intensity.